Relazione Progetto Hotel Excelsior

Corso: Applicazioni Web, Mobile e Cloud

Anno Accademico: 2025-2026

---

1. Introduzione

Il progetto consiste nella realizzazione di un sito web completo per la gestione di un hotel, denominato "Hotel Excelsior". L'applicazione permette a clienti anonimi di consultare le camere e il menu del ristorante, a clienti registrati di prenotare camere e cene con pagamento online (simulato), e a un amministratore di gestire camere, menu, prenotazioni e messaggi ricevuti dal form contatti.

L'obiettivo didattico è stato quello di mettere in pratica i principi di progettazione di applicazioni web moderne: separazione delle responsabilità, sicurezza, containerizzazione, integrazione continua, persistenza cloud e gestione degli errori.

---

2. Architettura del sistema

2.1 Architettura a due livelli

Il sistema è composto da due componenti principali che comunicano via HTTP:

- **Frontend**: Single Page Application sviluppata in React 18 con TypeScript, servita da un web server nginx.
- **Backend**: Web API RESTful scritta in ASP.NET Core 10, che gestisce tutta la logica applicativa e la persistenza dei dati.

I due componenti vengono distribuiti in container Docker separati, orchestrati da Docker Compose. nginx espone anche la porta pubblica 8080 e funge da reverse proxy per le chiamate API verso il container backend, evitando al client di gestire CORS in sviluppo.

2.2 Database

La persistenza è affidata a un database PostgreSQL ospitato su **Neon**, un servizio managed di PostgreSQL cloud. Questa scelta ha diversi vantaggi rispetto a un'istanza locale:

- Il database è condiviso tra tutti gli sviluppatori del team
- Non richiede installazione locale di PostgreSQL
- Si può accedere al DB anche da più ambienti (locale, staging, deploy)

Le migrazioni dello schema sono gestite da Entity Framework Core tramite il pattern *Code First*: il codice dell'entità è la fonte di verità, e il comando `dotnet ef migrations add` genera incrementalmente gli script SQL di modifica. L'applicazione all'avvio invoca `db.Database.Migrate()` e applica automaticamente le migrazioni pendenti al database, garantendo che lo schema sia sempre allineato al codice.

2.3 Clean Architecture nel backend

Il backend è organizzato in **quattro progetti .NET** che implementano i principi della Clean Architecture di Robert C. Martin:

Progetto	Responsabilità
`Hotel.Site.Core`	Entità di dominio (User, Room, RoomReservation, DinnerReservation, Payment, Contact, Menu). Nessuna dipendenza esterna.
`Hotel.Site.Application`	Contratti (interfacce `IUserService`, `IRoomReservationRepository`, `IEmailService`, `IUnitOfWork`) e implementazioni dei servizi applicativi. Definisce "cosa" fa l'applicazione.
`Hotel.Site.Infrastructure`	Implementazioni concrete: accesso al database con Entity Framework Core, invio email con MailKit, password hashing con BCrypt. Definisce "come" sono fatte le operazioni.
`Hotel.Site.Api`	Controller REST, DTO, configurazione del web host, dependency injection, middleware.

La regola fondamentale è che le **dipendenze puntano verso l'interno**: `Api` conosce `Application`, che conosce `Core`. `Infrastructure` implementa le interfacce definite in `Application`. Il dominio (`Core`) non conosce nulla del mondo esterno.

Il vantaggio pratico: se un domani volessi migrare da PostgreSQL a SQL Server, toccherei solo il progetto `Infrastructure`. Se volessi cambiare il provider email da Gmail a SendGrid, toccherei solo l'implementazione di `IEmailService`. Il dominio e la logica applicativa restano intatti.

2.4 Unit of Work e Repository

L'accesso al database avviene tramite il pattern **Repository** (`IUserRepository`, `IRoomReservationRepository`, ecc.) coordinato da un **Unit of Work** (`IUnitOfWork`). Questo pattern isola la logica dal provider specifico (EF Core) e permette di raggruppare più operazioni in una singola transazione atomica.

Esempio concreto nel metodo `Create` del `RoomReservationController`:

```
await _unitOfWork.BeginTransactionAsync();
try
{
    await _reservationService.AddRoomReservationAsync(reservation);
    await _paymentService.CreateForReservationAsync(reservation.IdRoomReservation, reservation.PrezzoTotale);
    await _unitOfWork.SaveChangesAsync();
    await _unitOfWork.CommitTransactionAsync();
}
catch (Exception ex)
{
    await _unitOfWork.RollbackTransactionAsync();
    return StatusCode(500, new { message = "Errore nella creazione della prenotazione" });
}
```

Se uno dei due passaggi fallisce, entrambi vengono annullati. Senza questa transazione esplicita, si rischiava di avere prenotazioni senza pagamento associato, con conseguenti problemi di consistenza.

---

3. Stack tecnologico

Layer	Tecnologia	Motivazione
Frontend	React 18 + TypeScript + Vite	Tipizzazione forte, hot reload veloce, community consolidata
Styling	TailwindCSS	Utility-first, evita CSS globali non gestibili
Routing	React Router v6	Standard de facto per React

Layer	Tecnologia	Motivazione
Autenticazione client	react-auth-kit	Gestione JWT integrata, supporta token storage sicuro
Backend	.NET 10 / ASP.NET Core	Performance elevate, ecosystem maturo, tipizzazione forte
ORM	Entity Framework Core 10 + Npgsql	Code First, migrations, supporto PostgreSQL
Database	PostgreSQL (Neon cloud)	Robusto, open source, managed
Email	MailKit	Standard moderno per SMTP in .NET
Auth server	JWT Bearer	Stateless, adatto a SPA + API separate
Password hashing	BCrypt.Net	Resistente a rainbow table e attacchi a forza bruta
Containerizzazione	Docker + Docker Compose	Environment riproducibile su ogni macchina
CI/CD	GitHub Actions	Integrato con il repo, gratuito
Testing	xUnit, Moq, FluentAssertions, EF InMemory	Standard .NET per unit + integration test

4. Funzionalità principali

4.1 Autenticazione e gestione utenti

Il sistema distingue due ruoli: **CLIENT** e **ADMIN**.

Endpoint:

- `POST /api/auth/register`: registrazione cliente. Password validata con regex (minuscole + maiuscole + numeri, min 8 caratteri), hashata con BCrypt (work factor 11) prima della memorizzazione.
- `POST /api/auth/login`: login cliente. Rifiuta utenti admin (403 Forbid) per evitare confusione di endpoint.
- `POST /api/auth/admin/login`: login amministratore. Rifiuta utenti non-admin.
- `POST /api/auth/forgot-password`: genera un token crittografico di 32 byte, ne salva l'hash SHA-256 nel DB con scadenza 30 minuti, invia email all'utente con il link di reset.
- `POST /api/auth/reset-password`: verifica il token, aggiorna la password.

Account lockout: dopo 5 login falliti consecutivi, l'account viene bloccato per 15 minuti. I contatori vengono resettati a login riuscito o dopo un reset password completato (l'utente ha dimostrato di possedere l'email).

Rate limiting: gli endpoint sensibili sono protetti dal `RateLimiter` nativo di ASP.NET Core con policy `FixedWindow` partizionate per IP:

- `login`: max 10 tentativi ogni 5 minuti
- `register` e `forgot-password`: max 3 ogni 15 minuti

Gli utenti eccedenti ricevono `429 Too Many Requests`.

4.2 Prenotazione camere e pagamento

Il flusso di prenotazione è implementato in più fasi con controlli espliciti a ciascun livello:

1. **Consultazione camere** (`/camere`): il visitatore seleziona date e numero ospiti. Il frontend chiama `GET /api/reservations/booked-rooms?checkIn=...&checkOut=...` che restituisce gli ID delle camere già prenotate nel periodo. Le camere occupate vengono mostrate grigie con badge "Non disponibile", il bottone di navigazione è disabilitato.
2. **Aggiunta al carrello**: sulla pagina di dettaglio (`/camere/{id}`), l'hook `useRoomAvailability` ripete il check in tempo reale alle date selezionate. Il bottone "Prenota ora" è disabilitato finché l'endpoint non conferma la disponibilità.
3. **Conferma del carrello**: alla pagina `/carrello`, l'hook `useCartAvailability` ricontrolla tutte le camere nel carrello (poiché altri utenti potrebbero aver prenotato nel frattempo). Gli item non più disponibili vengono evidenziati con overlay rosso e il bottone "Conferma prenotazione" viene disabilitato fino alla loro rimozione.
4. **Creazione prenotazione**: `POST /api/reservations` crea la `RoomReservation` e il relativo `Payment` (stato `IN_ATTESA`) in una **singola transazione atomica**. Se una delle due fallisce, entrambe vengono annullate.
5. **Conferma pagamento**: `POST /api/payments/{id}/confirm` riceve i dati della carta. Il numero viene validato con l'algoritmo di **Luhn**, la scadenza e il CVV sono controllati. Non viene mai memorizzato il PAN completo: solo le ultime 4 cifre. Il metodo è idempotente: se il pagamento è già completato, la seconda chiamata restituisce il record invariato senza effetti collaterali.

4.3 Gestione scadenza prenotazioni non pagate

Una prenotazione in stato `IN_ATTESA` senza pagamento completato **non blocca la camera indefinitamente**. La query `HasOverlappingReservationAsync` considera come "occupante" una prenotazione solo se:

- è in stato `CONFERMATO`, **oppure**
- ha un `Payment` in stato `COMPLETATO`, **oppure**
- è stata creata negli ultimi 15 minuti (grace period per completare il pagamento)

Dopo 15 minuti di inattività, la camera torna disponibile senza bisogno di job schedulati di pulizia.

4.4 Form contatti con notifica email

Qualsiasi visitatore può inviare un messaggio dalla pagina `/contatti`. Il messaggio viene salvato nella tabella `Contacts` e una email di notifica viene inviata all'amministratore via MailKit (SMTP Gmail con App Password).

L'invio email è **fire-and-forget** con gestione eccezioni isolata: un eventuale fallimento SMTP non blocca la risposta al client, l'errore viene solo loggato. L'utente vede la conferma immediatamente.

Dashboard admin (`/admin/contatti`): pagina protetta che mostra l'elenco paginato dei messaggi ricevuti con ricerca full-text.

4.5 Reset password con flusso sicuro

1. L'utente richiede il reset inserendo la sua email
2. Il sistema genera un token casuale di 32 byte encoded in base64url
3. Nel DB viene salvato solo l'**hash SHA-256** del token (non il token in chiaro)

4. L'utente riceve email con link `http://localhost:8080/reset-password?token=XXXX`
5. Clicca il link, imposta la nuova password
6. Il sistema calcola l'hash del token ricevuto, cerca l'utente corrispondente, verifica la scadenza (30 minuti), hasha la nuova password con BCrypt, invalida il token

Due accorgimenti di sicurezza:

- **Storage del solo hash**: una eventuale compromissione del DB non espone token utilizzabili per reset non autorizzati
- **Anti-enumerazione**: `POST /forgot-password` risponde sempre `200 OK` con messaggio generico, anche se l'email non è registrata, per prevenire attacchi che cercano di capire quali account esistono

---

5. Sicurezza

La sicurezza è stata trattata come requisito trasversale, non come afterthought.

5.1 Autenticazione e autorizzazione

- **JWT Bearer**: i token sono firmati con una chiave simmetrica HMAC-SHA256 di almeno 32 caratteri, conservata nel file `.env` locale e **non** nel repository. Ogni richiesta autenticata presenta il token nell'header `Authorization`.
- **Validazione completa**: Issuer, Audience, scadenza e firma sono verificati a ogni richiesta (`ValidateIssuer`, `ValidateAudience`, `ValidateLifetime`, `ValidateIssuerSigningKey`).
- **Attributi di autorizzazione**: `[Authorize]` su tutti gli endpoint che richiedono login; `[Authorize(Roles = "ADMIN")]` su quelli riservati all'amministratore.
- **Controlli di ownership**: endpoint come `GET /api/reservations/user/{userId}` o `PUT /api/users/{id}` verificano che il chiamante sia l'owner o un admin, restituendo `403 Forbid` altrimenti.

5.2 Protezione dei dati sensibili

- **Password utenti**: hashate con **BCrypt** (work factor 11). BCrypt è lento volutamente, rendendo impraticabili attacchi a forza bruta con hardware dedicato.
- **Dati carta di credito**: solo le ultime 4 cifre e il nome del titolare vengono memorizzati. Il PAN completo e il CVV non toccano mai il database. Il TransactionId è un codice generato lato backend.
- **Token di reset**: hashati con SHA-256 prima di essere salvati.
- **Secrets**: tutte le credenziali (connection string Neon, chiave JWT, password SMTP Gmail, URL frontend) sono esternalizzate in un file `.env` escluso dal versionamento tramite `.gitignore`. Un file `.env.example` committato fornisce il template per i nuovi sviluppatori.

5.3 Protezione anti-abuso

- **Rate limiting** sui path pubblici sensibili per mitigare spam e brute-force
- **Account lockout** dopo ripetuti login falliti
- **Anti-enumerazione email** sul reset password

- **Foreign key constraint**: la relazione tra `Payment` e `RoomReservation` è configurata con `DeleteBehavior.Cascade` (cancellando una prenotazione viene cancellato il relativo payment), mentre le relazioni con `User` e `Room` usano `Restrict` per preservare lo storico.

5.4 Robustezza infrastrutturale

- **DataProtection keys** persistite su volume Docker (`./dp-keys`), evitando che il rebuild del container invalidi token interni di ASP.NET Core (come quelli usati per la protezione di cookie anti-forgery).
- **Global exception handler**: un middleware centrale cattura tutte le eccezioni non gestite, logga lo stack trace completo lato server e restituisce al client un payload standardizzato in formato RFC 7807 (`application/problem+json`) senza esporre dettagli interni.

---

6. Qualità del codice

6.1 Validazione server-side

Tutti i DTO di request usano **Data Annotations** (`[Required]`, `[EmailAddress]`, `[StringLength]`, `[Range]`, `[RegularExpression]`).
L'attributo `[ApiController]` sui controller attiva la validazione automatica: se il body non rispetta i vincoli, il framework restituisce `400 Bad Request` con un payload strutturato che elenca i campi invalidi e gli errori. Questo elimina la necessità di validazione manuale nei metodi e garantisce coerenza.

6.2 Test automatici

È stato creato un progetto `Hotel.Site.Tests` con xUnit, che include 13 test distribuiti su 4 file:

- `PasswordHasherTests`: verifica hash + verify di BCrypt, assicurandosi che lo stesso input produca hash diversi (salt).
- `PaymentServiceTests`: testa il comportamento di `ConfirmAsync`, inclusa l'**idempotenza** (la seconda conferma non altera lo stato).
- `RoomReservationOverlapTests`: usa un database EF Core **InMemory** per testare che `HasOverlappingReservationAsync` rilevi correttamente sovrapposizioni, escluda prenotazioni annullate, e consideri correttamente date adiacenti.
- `ResetTokenTests`: verifica che i token generati siano url-safe, unici, e che l'hashing sia deterministico.

6.3 Continuous Integration

Il progetto include una pipeline GitHub Actions (`.github/workflows/ci.yml`) con due job che girano a ogni push e pull request:

- **Frontend job**: `npm ci` + `npm run lint` + `npm run build`. Garantisce che il codice frontend compili e superi i controlli di lint.
- **Backend job**: `dotnet restore` + `dotnet build` + `dotnet test`. Compila l'intera soluzione e lancia i 13 test xUnit. I risultati dei test vengono salvati come artifact.

Un push che rompe il build viene immediatamente segnalato sulla PR con un check rosso.

6.4 Documentazione API

Il backend espone **Swagger UI** su `/swagger`. Tutti i controller sono decorati con:

- Commenti XML `/// <summary>` che diventano le descrizioni degli endpoint
- Attributi `[ProducesResponseType]` che dichiarano i codici di risposta e i tipi del payload

Questo permette al frontend di sapere esattamente cosa aspettarsi da ogni endpoint e facilita il testing manuale tramite interfaccia web.

6.5 Paginazione

Gli endpoint admin che ritornano liste (`GET /api/reservations`, `GET /api/contacts`) supportano paginazione server-side tramite query string `?page=N&pageSize=M`. La risposta è tipizzata come `PagedResponse<T>` con `items`, `page`, `pageSize`, `totalItems`, `totalPages`. Il repository usa `.Skip().Take()` con `AsNoTracking()` per ottimizzare le query di sola lettura. Il frontend consuma correttamente il nuovo formato nei relativi hook (`useContacts`, `useAdminReservations`).

---

7. Scelte progettuali e trade-off

7.1 Neon vs PostgreSQL locale

Inizialmente il progetto usava un container PostgreSQL locale. È stato migrato a Neon per permettere lavoro collaborativo con un DB condiviso tra gli sviluppatori. Il container postgres è stato rimosso dal `docker-compose.yml`. Trade-off: dipendenza da un servizio esterno, ma beneficio enorme in produttività di team.

7.2 Transazioni EF Core esplicite

Invece di affidarsi al comportamento implicito di `SaveChangesAsync` (che raggruppa tutte le modifiche in una transazione), sono state introdotte `BeginTransactionAsync` / `CommitTransactionAsync` / `RollbackTransactionAsync` nell'`IUnitOfWork`. Questo permette di controllare esplicitamente i confini transazionali e gestire il rollback in caso di errori applicativi (non solo di persistenza).

7.3 Pagamento simulato

Il sistema non si integra con un payment provider reale (es. Stripe, PayPal). La conferma del pagamento esegue una simulazione con validazione Luhn/scadenza/CVV e genera un TransactionId fittizio.

Come si integrerebbe Stripe in produzione? Basterebbe creare un'interfaccia `IPaymentProvider` con un metodo `ChargeAsync(amount, cardToken)` e una implementazione `StripePaymentProvider` che chiama l'API Stripe. `PaymentService.ConfirmAsync` userebbe l'interfaccia iniettata via DI, senza cambiare la logica applicativa. Il frontend caricherebbe Stripe.js per tokenizzare la carta lato client, cosicché il PAN completo non toccherebbe mai il nostro backend (requisito PCI DSS).

7.4 Componenti React e layout condiviso

Le 4 dashboard admin (camere, menu, prenotazioni, contatti) condividono un header uniforme con navigation tabs, info utente loggato e bottone logout. Invece di duplicare il markup in ogni pagina, è stato creato un componente `AdminLayout` che usa `<Outlet />` di React Router: le rotte `/admin/*` sono raggruppate dentro questo layout in `App.tsx`. Risultato: l'header è scritto una volta sola, e navigare tra le tabs non lo fa "saltare" perché è esterno alle pagine.

7.5 Interfaccia email astratta

`IEmailService` ha un singolo metodo `SendAsync(to, subject, body)`. L'implementazione concreta `SmtpEmailService` si connette a Gmail tramite MailKit. Se un domani si volesse passare a SendGrid, Amazon SES, Postmark, ecc., basterebbe una nuova implementazione. I controller che consumano l'interfaccia (`AuthController`, `ContactController`) restano invariati.

---

8. Limitazioni note e sviluppi futuri

8.1 Test coverage

Attualmente ci sono 13 test, principalmente focalizzati sui domini critici (password hashing, pagamento, overlap, token). Una copertura più estesa richiederebbe:

- Integration test dei controller con `WebApplicationFactory` e `TestServer`
- Test di componenti React con Vitest + Testing Library
- Test end-to-end del flusso utente con Playwright o Cypress

8.2 Sistema di storage immagini

Le immagini delle camere vengono salvate sul filesystem del container API in `/app/wwwroot/uploads` (montato su un volume Docker). Questa soluzione funziona in locale ma non scala: in produzione con più istanze del backend, ciascuna avrebbe un filesystem separato. La soluzione corretta sarebbe usare uno storage object come **Cloudinary**, **AWS S3** o **Azure Blob Storage**, salvando nel DB solo l'URL dell'immagine.

8.3 Real-time notifications

Attualmente la pagina `/camere` non aggiorna dinamicamente la disponibilità: se un altro utente prenota mentre l'utente corrente sta guardando, il cambio si vede solo al refresh o cambiando le date. Una versione più sofisticata userebbe **SignalR** (il protocollo real-time di ASP.NET) per notificare via WebSocket ai client quando una camera viene prenotata.

8.4 Deploy in produzione

Il progetto gira in Docker Compose in locale. Per un deploy reale:

- Il backend andrebbe deployato su un PaaS come **Render** o **Railway** (entrambi hanno tier gratuiti per container .NET)
- Il frontend su **Vercel** o **Netlify** con build automatico da GitHub
- Il database è già su Neon, non serve migrare
- Servirebbero variabili d'ambiente di produzione separate, con chiavi JWT e SMTP diverse da quelle di dev

8.5 Multi-factor authentication

Il sistema prevede già una UI per OTP nel file `PasswordForm.tsx` frontend, ma non è stato implementato il backend corrispondente. Aggiungere 2FA tramite TOTP (es. Google Authenticator) o invio codice email sarebbe un plus importante per account admin.

---

9. Conclusioni

Il progetto realizza un'applicazione web completa coprendo tutti i layer di un'architettura enterprise moderna: presentazione, logica di business, persistenza, infrastruttura. Sono state adottate pratiche di sviluppo diffuse nell'industria: clean architecture, dependency injection, dependency inversion, pattern Repository e Unit of Work, containerizzazione, integrazione continua, testing automatizzato.

Particolare attenzione è stata dedicata alla **sicurezza** (hashing BCrypt, token hashati, anti-enumerazione, rate limiting, account lockout, secrets esternalizzati) e alla **robustezza** (transazioni atomiche, gestione scadenze, rollback, global exception handler).

Le scelte tecnologiche (.NET 10, React + TypeScript, Docker, GitHub Actions, PostgreSQL cloud) riflettono uno stack attualmente richiesto nel mercato del lavoro, ed è stato possibile sperimentare concretamente come questi pezzi si integrano per formare un'applicazione completa.

Il codice è organizzato per essere **mantenibile** (separazione netta tra layer, interfacce per ogni componente critico) ed **estensibile** (nuovi provider email, payment, storage possono essere aggiunti senza toccare la logica applicativa). Lo sviluppo in team di due persone è stato reso agevole dall'uso di un database condiviso cloud e da una procedura di setup documentata nel README.

Repository GitHub: https://github.com/riccardodilizio-art/Progetto-Applicazioni-Web-Mobile-e-Cloud

Demo locale: http://localhost:8080 (sito), http://localhost:8081/swagger (API)